

The Producer–Consumer Problem

A Bounded-Buffer Simulator using
POSIX Threads, Semaphores, and Mutexes

Topic 10 — Synchronization Problems

Group Members

1. Hoàng Anh – 202417093
2. Đặng Xuân Bách – 202417099
3. Nguyễn Phúc Minh – 202417163
4. Trần Đức Nam – 202417175

Abstract

This report presents the design, implementation, and empirical evaluation of a bounded-buffer simulator for the classic Producer–Consumer synchronization problem. The solution is built on POSIX threads using two counting semaphores together with a mutex, cleanly separating *condition synchronization* (blocking on a full or empty buffer) from *mutual exclusion* (protecting the shared buffer state). We describe the producer and consumer algorithms, the ordering of operations that provably avoids deadlock, and a sentinel-based protocol for clean termination. The implementation is exercised under three speed regimes—balanced, fast-producer, and fast-consumer—and a built-in self-check confirms correctness in every case (items produced equal items consumed and the buffer ends empty), while the measured wait counts demonstrate the back-pressure a bounded buffer is expected to provide. The result is a correct, deadlock-free, and starvation-free reference implementation suitable for teaching the fundamental synchronization primitives of a modern operating system.

Contents

1	Introduction	1
1.1	Overview of the Topic	1
1.2	Problem Statement	1
2	Main Content	3
2.1	Chosen Solution and Algorithms	3
2.2	Analysis and Comparison with Other Solutions	4
2.3	Program Description	6
2.3.1	Input and Output Data	6
2.3.2	Key Functions and Their Purposes	7
3	Program Interface	9
3.1	Option 1 — Balanced Scenario	9
3.2	Option 2 — Fast Producer Scenario	12
3.3	Option 3 — Fast Consumer Scenario	13
3.4	Option 4 — Help	16
3.5	Option 0 — Exit	16
4	Conclusion	18
5	Teamwork	19
6	References	20

1 Introduction

1.1 Overview of the Topic

Modern operating systems run many activities at once. Threads and processes execute concurrently and frequently need to exchange data through a region of shared memory. Whenever several execution flows read and write the same memory without coordination, the result of the program becomes dependent on the unpredictable order in which the operating-system scheduler interleaves them. This phenomenon is called a *race condition*, and the section of code that accesses shared state is called a *critical section*. Process and thread synchronization is the branch of operating-system theory concerned with disciplining access to critical sections so that concurrent programs remain correct.

The **Producer–Consumer problem** (also known as the *bounded-buffer problem*) is the classic teaching example of this challenge. It was introduced by Edsger W. Dijkstra [3] and remains one of the canonical synchronization problems studied in every operating-systems curriculum [1, 2]. The scenario is simple to state: one or more *producer* threads generate data items and place them into a shared buffer of fixed size, while one or more *consumer* threads remove items from that buffer and process them. The buffer has finite capacity, so the two groups must cooperate:

- A producer must **wait** when the buffer is full, instead of overwriting unconsumed data.
- A consumer must **wait** when the buffer is empty, instead of reading a slot that holds no valid item.
- Producers and consumers must never touch the buffer’s internal bookkeeping (head, tail, count) at the same time.

Although the statement is short, a correct solution must simultaneously guarantee *mutual exclusion* over the shared buffer and correct *condition synchronization* (blocking when full or empty). It therefore exercises every fundamental synchronization primitive at once, which is exactly why it is such a valuable exercise.

1.2 Problem Statement

The project aims to build a simulator that demonstrates a correct, deadlock-free, and starvation-free solution to the bounded-buffer Producer–Consumer problem, and that measures the behaviour of the system under different relative speeds of producers and consumers.

Concretely, the program must solve the following sub-problems:

1. **Mutual exclusion.** Updates to the shared circular buffer (inserting an item, removing an item, advancing the head/tail indices and the element count) must be atomic so that no two threads ever corrupt the buffer.
2. **Condition synchronization.** Producers must block while the buffer is full and consumers

must block while it is empty, *without* resorting to busy-waiting (spinning), which wastes CPU cycles.

3. **No deadlock.** The locking discipline must be designed so that the system can never reach a state in which every thread is waiting forever.
4. **Clean termination.** When all producers have finished generating their items, every consumer must be told to stop in an orderly way, and all threads must be joined and all resources released.
5. **Observability.** The program must report measurable evidence of correctness (items produced equals items consumed, buffer empty at the end) together with performance metrics (execution time, throughput, and how often each side had to block).

2 Main Content

2.1 Chosen Solution and Algorithms

The implementation follows the classical **semaphore-and-mutex** solution [3, 7], which is the standard production-grade approach for this problem. Three synchronization objects guard a single shared circular buffer:

Object	Type	Purpose
empty	Counting semaphore	Counts the number of <i>free</i> slots. Initialised to the buffer capacity. A producer waits on it before inserting.
full	Counting semaphore	Counts the number of <i>occupied</i> slots. Initialised to 0. A consumer waits on it before removing.
mutex	Binary mutex	Protects the buffer’s internal state (items , head , tail , count) so that only one thread mutates it at a time.

The two counting semaphores handle *condition synchronization* (“is there room?” / “is there an item?”), while the mutex handles *mutual exclusion*. Separating these two concerns is what makes the solution both correct and efficient: a thread only ever holds the mutex for the few instructions needed to touch the buffer, and blocking on a full/empty condition happens *outside* the mutex so a waiting producer never locks out a consumer.

Producer algorithm

```

loop for each item to produce:
    sleep(random delay)           // simulate work
    wait(empty)                   // block while buffer is full
    lock(mutex)                   // >> enter critical section
        items[tail] = item
        tail = (tail + 1) mod capacity
        count = count + 1
    unlock(mutex)                 // << leave critical section
    signal(full)                  // announce a new item

```

Consumer algorithm

```

loop forever:
    wait(full)                    // block while buffer is empty
    lock(mutex)                   // >> enter critical section
        item = items[head]
        head = (head + 1) mod capacity
        count = count - 1
    unlock(mutex)                 // << leave critical section
    signal(empty)                 // announce a free slot
    if item == TERMINATION_ITEM: stop
    else: process(item); sleep(random delay)

```

Ordering of wait operations (deadlock avoidance)

A crucial design point is that each thread first waits on the *counting* semaphore and only then acquires the mutex. The reverse order — acquiring the mutex *before* checking whether the buffer is full/empty — would let a producer fall asleep on a full buffer while still holding the mutex, permanently locking out every consumer. By waiting on **empty/full** *before* taking the mutex, the implementation provably avoids this deadlock.

Termination via sentinel items

Because consumers loop forever waiting for items, they need an explicit stop signal. After every producer has finished and been joined, the main simulation thread inserts one special *sentinel* value (`TERMINATION_ITEM`, defined as `-1`) into the buffer for each consumer. A consumer that dequeues the sentinel returns immediately instead of processing it. Because exactly one sentinel is sent per consumer and the sentinels travel through the same synchronized buffer as ordinary items, every consumer is guaranteed to receive exactly one and shut down cleanly — with no busy-waiting and no lost items.

2.2 Analysis and Comparison with Other Solutions

The chosen design was compared conceptually against the other well-known approaches to the bounded-buffer problem:

Approach	Strengths	Weaknesses
Busy-wait / spinlock only	Simple to write; no blocking primitives needed.	Wastes CPU spinning; does not scale; effectively unusable for real workloads.
Semaphore + mutex (this project)	No busy-waiting; clean separation of mutual exclusion and condition synchronization; maps directly to POSIX primitives; high throughput.	Programmer must order operations carefully or risk deadlock.
Monitor / condition variables	High-level abstraction; the lock and the wait conditions are bundled together; safer for complex predicates.	Requires re-checking the predicate in a <code>while</code> loop because of spurious wake-ups or changes to the condition before the awakened thread reacquires the lock.
Lock-free SPSC ring buffer	Very low latency and high throughput for one producer and one consumer; avoids mutexes and operating-system context switches; benefits from cache-line padding and acquire–release memory ordering.	Limited to a single-producer, single-consumer topology; difficult to implement correctly; spinning may waste CPU when the buffer remains empty or full.
Lock-free MPMC ring buffer	Supports multiple producers and multiple consumers without a global mutex; atomic operations and per-slot sequence numbers can provide high throughput on multi-core systems.	Much more complex to implement; requires Compare-and-Swap operations, careful memory ordering, and protection against ABA-related problems; contention may cause repeated retries and high CPU usage.

The semaphore-and-mutex design was selected as the primary Linux implementation because POSIX exposes counting semaphores (`sem_t`) [4] directly, making the mapping from theory to code exact and the runtime cost minimal. The buffer’s mutual exclusion is provided by a POSIX mutex [6], and all worker threads are created and joined through the POSIX threads API [5]. For portability, the program also ships a *condition-variable fallback* (used automatically on macOS, whose kernel does not support unnamed POSIX semaphores), demonstrating the monitor-style approach [2] within the same codebase.

Empirical correctness and efficiency

Every run ends with a built-in self-check confirming that the number of items produced equals the number expected, that items consumed equals items produced, and that the buffer is empty. The program also reports the execution time, throughput, and the number of times producers and consumers had to block. A representative no-delay stress configuration (8-slot buffer, 10 producers, 10 consumers, 1,000 items each — 100,000 items total) completes with all correctness checks passing and sustains a throughput on the order of 10^5 items per second, confirming both correctness and efficiency under heavy contention.

2.3 Program Description

The program is written in portable C11 and organised into small, single-responsibility modules. The source layout is:

File	Responsibility
<code>main.c</code>	Program entry point; hands control to the CLI.
<code>cli.c</code>	Interactive text menu, input validation, scenario dispatch.
<code>scenario.c</code>	Defines the three predefined scenarios and prints their configuration.
<code>simulation.c</code>	Orchestrates one simulation: validates config, spawns threads, joins them, sends sentinels, prints results.
<code>bounded_buffer.c</code>	The shared circular buffer with <code>put/get</code> and all synchronization.
<code>worker.c</code>	The producer and consumer thread routines.
<code>statistics.c</code>	Thread-safe counters for produced/consumed items and wait events.
<code>logger.c</code>	Time-stamped, mutex-protected event logging.

2.3.1 Input and Output Data

Input. The program is interactive and takes no command-line arguments or input files. The user supplies, through the terminal:

- a menu choice (0–4) selecting a scenario, the help screen, or exit;
- a `y/n` confirmation before a simulation starts;
- a keypress to return to the menu after a run.

Internally, each scenario is represented by a `SimulationConfig` structure carrying the buffer capacity, the number of producers and consumers, the number of items per producer, the minimum/maximum producer and consumer delays, and the logging mode.

Output. During a run the program streams a synchronized event log; each line is time-stamped (milliseconds since start) and shows the acting thread, the item produced or consumed, the

resulting buffer occupancy, and whether the thread had to block. After the run it prints a results block:

```
Simulation Results
=====
Expected data items: 20
Total items produced: 20
Total items consumed: 20
Items remaining: 0
Producer wait events: ...
Consumer wait events: ...
Execution time: ... seconds
Throughput: ... items/second
Correctness checks: PASS
```

2.3.2 Key Functions and Their Purposes

Bounded buffer (`bounded_buffer.c`).

- `buffer_init` — allocates the item array and initialises the mutex and the `empty/full` semaphores (with full error-rollback if any step fails).
- `buffer_put` — waits on `empty`, locks the mutex, inserts the item at `tail`, releases the mutex, and signals `full`. Reports whether the producer blocked and the new occupancy.
- `buffer_get` — the symmetric operation: waits on `full`, locks the mutex, removes from `head`, releases the mutex, and signals `empty`.
- `buffer_destroy` — destroys all synchronization objects and frees memory.

Workers (`worker.c`).

- `producer_run` — the producer thread body: for each item, sleeps a random delay, builds a unique item value, calls `buffer_put`, and records statistics and a log line.
- `consumer_run` — the consumer thread body: repeatedly calls `buffer_get`, stops on the sentinel, otherwise records statistics, logs, and sleeps. A fast XOR-shift random generator seeded per thread produces the delays.

Simulation control (`simulation.c`).

- `simulation_run` — the heart of the program: validates the configuration, initialises the buffer/statistics/logger, creates the consumer and producer threads, joins the producers, sends one sentinel per consumer, joins the consumers, computes the final correctness verdict, and prints the results.
- `validate_config` — rejects impossible or overflow-prone configurations before any thread is created.

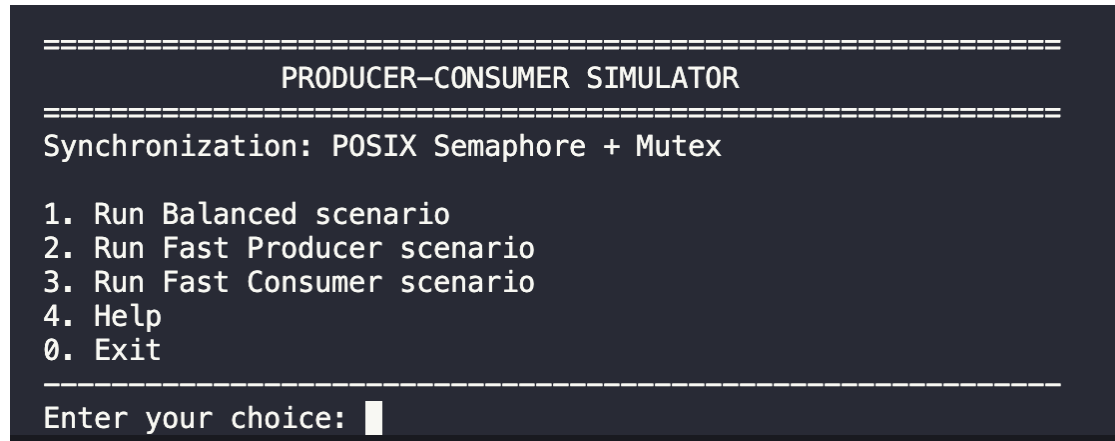
- `send_termination_items` / `join_threads` / `print_results` — helpers for orderly shut-down, thread collection, and reporting.

Statistics (`statistics.c`). A mutex-protected record of total items produced and consumed and the number of producer/consumer wait events. `statistics_snapshot` returns a consistent copy for the final report.

Logger (`logger.c`). `logger_event` prints a single, atomically-locked, time-stamped line so that concurrent threads never interleave their output mid-line.

3 Program Interface

The program provides an interactive **command-line interface (CLI)** rather than a graphical one, which keeps the focus on the synchronization behaviour and makes the event log easy to read. The program is started with `make run`, after which the main menu is displayed (Figure 1).



```
=====
                          PRODUCER-CONSUMER SIMULATOR
=====
Synchronization: POSIX Semaphore + Mutex

1. Run Balanced scenario
2. Run Fast Producer scenario
3. Run Fast Consumer scenario
4. Help
0. Exit

-----
Enter your choice: █
```

Figure 1: The main menu shown on startup. The header confirms the synchronization strategy in use (POSIX semaphore + mutex) and lists the five available choices.

The menu accepts a single digit:

- **1 / 2 / 3** — run the Balanced, Fast Producer, or Fast Consumer scenario;
- **4** — display the help screen;
- **0** — exit the program.

The CLI validates every keystroke: numeric choices outside the menu range and malformed `y/n` answers are rejected with a clear message and re-prompted, so the program cannot be driven into an invalid state. The three scenarios all share a buffer capacity of 5, two producers, two consumers, and ten items per producer (so $2 \times 10 = 20$ items are expected each run); they differ only in the relative speed of producers and consumers. Each of the five menu options is described below together with its on-screen output.

3.1 Option 1 — Balanced Scenario

When the user enters 1, the program first prints the scenario configuration and asks for confirmation (Figure 2). In this scenario producers and consumers use the same delay range (200–400 ms), so neither side dominates.

```
=====
                        PRODUCER-CONSUMER SIMULATOR
=====
Synchronization: POSIX Semaphore + Mutex

1. Run Balanced scenario
2. Run Fast Producer scenario
3. Run Fast Consumer scenario
4. Help
0. Exit

-----
Enter your choice: 1

Scenario: Balanced
-----
Buffer capacity:      5
Number of producers:  2
Number of consumers:  2
Items per producer:  10
Producer delay:       200-400 ms
Consumer delay:       200-400 ms
Logging mode:         Detailed

Start simulation? [y/n]: █
```

Figure 2: Option 1 — the Balanced scenario configuration and the confirmation prompt.

After the user confirms with `y`, the simulation runs and streams a time-stamped event log, followed by the results block (Figure 3). Because the two sides run at matching speeds, the buffer occupancy hovers near empty and only a handful of waits occur: in the run shown, producers blocked **0** times and consumers blocked only **4** times, and the correctness check passes (20 produced, 20 consumed, 0 remaining).

```

Start simulation? [y/n]: y
[000219 ms] [Producer 2 ] Produced item 200001 | Buffer: 1/5
[000219 ms] [Consumer 1 ] Consumed item 200001 | Buffer: 0/5 | waited
[000371 ms] [Producer 1 ] Produced item 100001 | Buffer: 1/5
[000371 ms] [Consumer 2 ] Consumed item 100001 | Buffer: 0/5 | waited
[000590 ms] [Producer 2 ] Produced item 200002 | Buffer: 1/5
[000611 ms] [Consumer 1 ] Consumed item 200002 | Buffer: 0/5
[000634 ms] [Producer 1 ] Produced item 100002 | Buffer: 1/5
[000728 ms] [Consumer 2 ] Consumed item 100002 | Buffer: 0/5
[000948 ms] [Producer 1 ] Produced item 100003 | Buffer: 1/5
[000948 ms] [Consumer 1 ] Consumed item 100003 | Buffer: 0/5 | waited
[000970 ms] [Producer 2 ] Produced item 200003 | Buffer: 1/5
[001060 ms] [Consumer 2 ] Consumed item 200003 | Buffer: 0/5
[001233 ms] [Producer 2 ] Produced item 200004 | Buffer: 1/5
[001233 ms] [Consumer 1 ] Consumed item 200004 | Buffer: 0/5 | waited
[001329 ms] [Producer 1 ] Produced item 100004 | Buffer: 1/5
[001393 ms] [Consumer 2 ] Consumed item 100004 | Buffer: 0/5
[001449 ms] [Producer 2 ] Produced item 200005 | Buffer: 1/5
[001548 ms] [Producer 1 ] Produced item 100005 | Buffer: 2/5
[001551 ms] [Consumer 1 ] Consumed item 200005 | Buffer: 1/5
[001677 ms] [Consumer 2 ] Consumed item 100005 | Buffer: 0/5
[001695 ms] [Producer 2 ] Produced item 200006 | Buffer: 1/5
[001791 ms] [Producer 1 ] Produced item 100006 | Buffer: 2/5
[001897 ms] [Producer 2 ] Produced item 200007 | Buffer: 3/5
[001942 ms] [Consumer 1 ] Consumed item 200006 | Buffer: 2/5
[001985 ms] [Consumer 2 ] Consumed item 100006 | Buffer: 1/5
[002093 ms] [Producer 1 ] Produced item 100007 | Buffer: 2/5
[002231 ms] [Consumer 1 ] Consumed item 200007 | Buffer: 1/5
[002286 ms] [Producer 2 ] Produced item 200008 | Buffer: 2/5
[002322 ms] [Consumer 2 ] Consumed item 100007 | Buffer: 1/5
[002440 ms] [Producer 1 ] Produced item 100008 | Buffer: 2/5
[002487 ms] [Consumer 1 ] Consumed item 200008 | Buffer: 1/5
[002602 ms] [Producer 2 ] Produced item 200009 | Buffer: 2/5
[002662 ms] [Producer 1 ] Produced item 100009 | Buffer: 3/5
[002713 ms] [Consumer 1 ] Consumed item 100008 | Buffer: 2/5
[002723 ms] [Consumer 2 ] Consumed item 200009 | Buffer: 1/5
[002911 ms] [Producer 1 ] Produced item 100010 | Buffer: 2/5
[002952 ms] [Producer 2 ] Produced item 200010 | Buffer: 3/5
[002990 ms] [Consumer 2 ] Consumed item 100009 | Buffer: 4/5
[003062 ms] [Consumer 1 ] Consumed item 100010 | Buffer: 3/5
[003240 ms] [Consumer 2 ] Consumed item 200010 | Buffer: 2/5

Simulation Results
=====
Expected data items: 20
Total items produced: 20
Total items consumed: 20
Items remaining: 0
Producer wait events: 0
Consumer wait events: 4
Execution time: 3.554 seconds
Throughput: 5.63 items/second
Correctness checks: PASS

```

Figure 3: Option 1 — Balanced run: the synchronized event log and the final results. Each line shows the timestamp, the acting thread, the item, the resulting buffer occupancy, and a **waited** marker when the thread had to block.

3.2 Option 2 — Fast Producer Scenario

Entering 2 selects the Fast Producer scenario (Figure 4), in which producers are fast (50–100 ms) and consumers are slow (400–600 ms).

```
=====
                          PRODUCER-CONSUMER SIMULATOR
=====
Synchronization: POSIX Semaphore + Mutex

1. Run Balanced scenario
2. Run Fast Producer scenario
3. Run Fast Consumer scenario
4. Help
0. Exit

Enter your choice: 2

Scenario: Fast Producer
=====
Buffer capacity:      5
Number of producers:  2
Number of consumers:  2
Items per producer:   10
Producer delay:       50–100 ms
Consumer delay:       400–600 ms
Logging mode:         Detailed

Start simulation? [y/n]: █
```

Figure 4: Option 2 — the Fast Producer scenario configuration.

The run output (Figure 5) clearly demonstrates the **empty** semaphore at work: the buffer quickly fills to its capacity (**Buffer: 5/5**) and producers are repeatedly forced to wait. In the run shown, **producer wait events rose to 13** while consumer waits fell to 2 — the mirror image of the Balanced run — yet correctness is still preserved (20/20, 0 remaining). This is exactly the back-pressure a bounded buffer is supposed to provide: fast producers cannot overrun slow consumers.

```

Start simulation? [y/n]: y
[000055 ms] [Producer 2 ] Produced item 200001 | Buffer: 1/5
[000055 ms] [Consumer 1 ] Consumed item 200001 | Buffer: 0/5 | waited
[000081 ms] [Producer 1 ] Produced item 100001 | Buffer: 1/5
[000081 ms] [Consumer 2 ] Consumed item 100001 | Buffer: 0/5 | waited
[000110 ms] [Producer 2 ] Produced item 200002 | Buffer: 1/5
[000167 ms] [Producer 1 ] Produced item 100002 | Buffer: 2/5
[000182 ms] [Producer 2 ] Produced item 200003 | Buffer: 3/5
[000257 ms] [Producer 1 ] Produced item 100003 | Buffer: 4/5
[000258 ms] [Producer 2 ] Produced item 200004 | Buffer: 5/5
[000500 ms] [Consumer 1 ] Consumed item 200002 | Buffer: 4/5
[000500 ms] [Producer 1 ] Produced item 100004 | Buffer: 5/5 | waited
[000639 ms] [Consumer 2 ] Consumed item 100002 | Buffer: 4/5
[000639 ms] [Producer 2 ] Produced item 200005 | Buffer: 5/5 | waited
[001011 ms] [Consumer 1 ] Consumed item 200003 | Buffer: 4/5
[001011 ms] [Producer 1 ] Produced item 100005 | Buffer: 5/5 | waited
[001228 ms] [Consumer 2 ] Consumed item 100003 | Buffer: 4/5
[001228 ms] [Producer 2 ] Produced item 200006 | Buffer: 5/5 | waited
[001528 ms] [Consumer 1 ] Consumed item 200004 | Buffer: 4/5
[001529 ms] [Producer 1 ] Produced item 100006 | Buffer: 5/5 | waited
[001716 ms] [Consumer 2 ] Consumed item 100004 | Buffer: 4/5
[001716 ms] [Producer 2 ] Produced item 200007 | Buffer: 5/5 | waited
[001988 ms] [Consumer 1 ] Consumed item 200005 | Buffer: 4/5
[001988 ms] [Producer 1 ] Produced item 100007 | Buffer: 5/5 | waited
[002283 ms] [Producer 2 ] Produced item 200008 | Buffer: 5/5 | waited
[002283 ms] [Consumer 2 ] Consumed item 100005 | Buffer: 4/5
[002528 ms] [Consumer 1 ] Consumed item 200006 | Buffer: 4/5
[002528 ms] [Producer 1 ] Produced item 100008 | Buffer: 5/5 | waited
[002839 ms] [Consumer 2 ] Consumed item 100006 | Buffer: 4/5
[002840 ms] [Producer 2 ] Produced item 200009 | Buffer: 5/5 | waited
[002937 ms] [Consumer 1 ] Consumed item 200007 | Buffer: 4/5
[002938 ms] [Producer 1 ] Produced item 100009 | Buffer: 5/5 | waited
[003367 ms] [Consumer 2 ] Consumed item 100007 | Buffer: 4/5
[003367 ms] [Producer 2 ] Produced item 200010 | Buffer: 5/5 | waited
[003415 ms] [Consumer 1 ] Consumed item 200008 | Buffer: 4/5
[003415 ms] [Producer 1 ] Produced item 100010 | Buffer: 5/5 | waited
[003918 ms] [Consumer 1 ] Consumed item 100008 | Buffer: 4/5
[003965 ms] [Consumer 2 ] Consumed item 200009 | Buffer: 4/5
[004337 ms] [Consumer 1 ] Consumed item 100009 | Buffer: 4/5
[004561 ms] [Consumer 2 ] Consumed item 200010 | Buffer: 3/5
[004860 ms] [Consumer 1 ] Consumed item 100010 | Buffer: 2/5

Simulation Results
=====
Expected data items: 20
Total items produced: 20
Total items consumed: 20
Items remaining: 0
Producer wait events: 13
Consumer wait events: 2
Execution time: 5.398 seconds
Throughput: 3.71 items/second
Correctness checks: PASS

Press Enter to return to the main menu...

```

Figure 5: Option 2 — Fast Producer run. Note the recurring Buffer: 5/5 | waited lines: producers block on a full buffer until a consumer frees a slot.

3.3 Option 3 — Fast Consumer Scenario

Entering 3 selects the Fast Consumer scenario (Figure 6), the opposite case: slow producers (400–600 ms) and fast consumers (50–100 ms).


```
=====
                          PRODUCER–CONSUMER SIMULATOR
=====
Synchronization: POSIX Semaphore + Mutex

1. Run Balanced scenario
2. Run Fast Producer scenario
3. Run Fast Consumer scenario
4. Help
0. Exit
=====
Enter your choice: 3

Scenario: Fast Consumer
=====
Buffer capacity:      5
Number of producers:  2
Number of consumers:  2
Items per producer:   10
Producer delay:       400–600 ms
Consumer delay:       50–100 ms
Logging mode:         Detailed

Start simulation? [y/n]: █
```

Figure 6: Option 3 — the Fast Consumer scenario configuration and confirmation prompt.

Here the **full** semaphore dominates (Figure 7): the buffer is almost always empty (**Buffer: 0/5**), and *every* consume operation is marked **waited** because consumers outrun producers and must block until an item appears. The results confirm this with **20 consumer wait events** (one per item) and 0 producer waits. Correctness again holds (20/20, 0 remaining), showing that the solution is correct under all three speed regimes.

```

Start simulation? [y/n]: y
[000414 ms] [Producer 1 ] Produced item 100001 | Buffer: 1/5
[000414 ms] [Consumer 1 ] Consumed item 100001 | Buffer: 0/5 | waited
[000423 ms] [Producer 2 ] Produced item 200001 | Buffer: 1/5
[000423 ms] [Consumer 2 ] Consumed item 200001 | Buffer: 0/5 | waited
[000874 ms] [Producer 1 ] Produced item 100002 | Buffer: 1/5
[000874 ms] [Consumer 2 ] Consumed item 100002 | Buffer: 0/5 | waited
[000997 ms] [Producer 2 ] Produced item 200002 | Buffer: 1/5
[000997 ms] [Consumer 1 ] Consumed item 200002 | Buffer: 0/5 | waited
[001374 ms] [Producer 1 ] Produced item 100003 | Buffer: 1/5
[001374 ms] [Consumer 2 ] Consumed item 100003 | Buffer: 0/5 | waited
[001561 ms] [Producer 2 ] Produced item 200003 | Buffer: 1/5
[001561 ms] [Consumer 1 ] Consumed item 200003 | Buffer: 0/5 | waited
[001872 ms] [Producer 1 ] Produced item 100004 | Buffer: 1/5
[001872 ms] [Consumer 2 ] Consumed item 100004 | Buffer: 0/5 | waited
[002050 ms] [Producer 2 ] Produced item 200004 | Buffer: 1/5
[002050 ms] [Consumer 1 ] Consumed item 200004 | Buffer: 0/5 | waited
[002379 ms] [Producer 1 ] Produced item 100005 | Buffer: 1/5
[002379 ms] [Consumer 2 ] Consumed item 100005 | Buffer: 0/5 | waited
[002531 ms] [Producer 2 ] Produced item 200005 | Buffer: 1/5
[002531 ms] [Consumer 1 ] Consumed item 200005 | Buffer: 0/5 | waited
[002895 ms] [Producer 1 ] Produced item 100006 | Buffer: 1/5
[002895 ms] [Consumer 2 ] Consumed item 100006 | Buffer: 0/5 | waited
[003118 ms] [Producer 2 ] Produced item 200006 | Buffer: 1/5
[003118 ms] [Consumer 1 ] Consumed item 200006 | Buffer: 0/5 | waited
[003444 ms] [Producer 1 ] Produced item 100007 | Buffer: 1/5
[003444 ms] [Consumer 2 ] Consumed item 100007 | Buffer: 0/5 | waited
[003600 ms] [Producer 2 ] Produced item 200007 | Buffer: 1/5
[003600 ms] [Consumer 1 ] Consumed item 200007 | Buffer: 0/5 | waited
[003876 ms] [Producer 1 ] Produced item 100008 | Buffer: 1/5
[003876 ms] [Consumer 2 ] Consumed item 100008 | Buffer: 0/5 | waited
[004153 ms] [Producer 2 ] Produced item 200008 | Buffer: 1/5
[004154 ms] [Consumer 1 ] Consumed item 200008 | Buffer: 0/5 | waited
[004388 ms] [Producer 1 ] Produced item 100009 | Buffer: 1/5
[004388 ms] [Consumer 2 ] Consumed item 100009 | Buffer: 0/5 | waited
[004669 ms] [Producer 2 ] Produced item 200009 | Buffer: 1/5
[004669 ms] [Consumer 1 ] Consumed item 200009 | Buffer: 0/5 | waited
[004809 ms] [Producer 1 ] Produced item 100010 | Buffer: 1/5
[004809 ms] [Consumer 2 ] Consumed item 100010 | Buffer: 0/5 | waited
[005096 ms] [Producer 2 ] Produced item 200010 | Buffer: 1/5
[005096 ms] [Consumer 1 ] Consumed item 200010 | Buffer: 0/5 | waited

Simulation Results
=====
Expected data items: 20
Total items produced: 20
Total items consumed: 20
Items remaining: 0
Producer wait events: 0
Consumer wait events: 20
Execution time: 5.166 seconds
Throughput: 3.87 items/second
Correctness checks: PASS

Press Enter to return to the main menu...

```

Figure 7: Option 3 — Fast Consumer run. Almost every consumer line carries the **waited** marker, and the buffer rarely holds more than one item.

The three scenarios together provide a compact, measurable comparison of how producer/consumer speed affects blocking:

Scenario	Producer waits	Consumer waits	Observed buffer behaviour
Balanced	0	4	Stays near empty; few blocks on either side.
Fast Producer	13	2	Fills to 5/5; producers block on empty.
Fast Consumer	0	20	Drains to 0/5; consumers block on full.

3.4 Option 4 — Help

Entering 4 prints a short help screen (Figure 8) explaining the purpose of the scenarios and the synchronization primitives, then returns to the menu.

```

=====
                        PRODUCER–CONSUMER SIMULATOR
=====
Synchronization: POSIX Semaphore + Mutex

1. Run Balanced scenario
2. Run Fast Producer scenario
3. Run Fast Consumer scenario
4. Help
0. Exit

-----
Enter your choice: 4

HELP
-----
Choose one of the three predefined scenarios to observe how
producer and consumer speeds affect a bounded circular buffer.
Threads synchronize with counting semaphores and a mutex.
Detailed logs show buffer occupancy and blocking events.

Press Enter to return to the main menu...

```

Figure 8: Option 4 — the help screen.

3.5 Option 0 — Exit

Entering 0 prints a farewell message and terminates the program cleanly, returning control to the shell (Figure 9).

```
=====
                        PRODUCER-CONSUMER SIMULATOR
=====
Synchronization: POSIX Semaphore + Mutex

1. Run Balanced scenario
2. Run Fast Producer scenario
3. Run Fast Consumer scenario
4. Help
0. Exit
=====
Enter your choice: 0
Goodbye.
```

Figure 9: Option 0 — exiting the simulator.

4 Conclusion

This project set out to build a correct, deadlock-free, and starvation-free solution to the bounded-buffer Producer–Consumer problem, and to measure how the system behaves as the relative speeds of producers and consumers change. Those goals were met. The chosen semaphore-and-mutex design maps the theory of condition synchronization and mutual exclusion directly onto POSIX primitives: two counting semaphores (`empty` and `full`) block threads when the buffer is full or empty, while a single mutex keeps every update to the shared circular buffer atomic. Waiting on the counting semaphores *before* acquiring the mutex is what guarantees the design can never deadlock, and a sentinel-based protocol lets every consumer shut down cleanly once production is complete, with no busy-waiting and no lost items.

The empirical results confirm the implementation across all three scenarios. In every run the built-in self-check passed—items produced equalled items consumed and the buffer ended empty—while the recorded wait counts told the expected story: a balanced workload produced very few blocks on either side, a fast producer was repeatedly forced to wait on a full buffer, and a fast consumer was repeatedly forced to wait on an empty one. This is exactly the back-pressure that a bounded buffer is designed to provide, and the high-contention stress test (10^5 items per second) showed that the approach remains both correct and efficient under load.

Beyond the working program, the exercise demonstrates why the Producer–Consumer problem remains a cornerstone of operating-systems education: a deceptively short problem statement forces a solution to juggle mutual exclusion, condition synchronization, deadlock avoidance, and orderly termination simultaneously. Natural extensions of this work include supporting multiple independent buffers, adding priority or fairness policies among threads, and comparing the semaphore-based implementation against a pure monitor/condition-variable design on a wider range of hardware.

5 Teamwork

The work was divided so that each member owned a coherent part of the system while collaborating on design and testing. The table below summarises the contributions.

Member	Student ID	Main Contributions
Hoàng Anh	202417093	Designed and implemented the shared bounded buffer (<code>bounded_buffer.c</code>): the circular buffer, the semaphore/mutex synchronization logic, and the macOS condition-variable fallback.
Đặng Xuân Bách	202417099	Implemented the producer and consumer worker routines (<code>worker.c</code>), the per-thread random delay generator, and the sentinel-based termination protocol.
Nguyễn Phúc Minh	202417163	Built the simulation orchestration (<code>simulation.c</code>), the thread-safe statistics module, and the synchronized logger; integrated the modules and prepared the report.
Trần Đức Nam	202417175	Developed the command-line interface and scenario definitions (<code>cli.c</code> , <code>scenario.c</code>), wrote the automated test suite, and ran the correctness/performance evaluation.

All members participated in the overall design discussions, reviewed each other’s code, and verified the final results together. Every member understands the complete project.

6 References

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018. ISBN: 978-1-119-32091-3.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015. ISBN: 978-0-13-359162-0.
- [3] E. W. Dijkstra, “Cooperating sequential processes,” Technological University, Eindhoven, The Netherlands, EWD-123, 1968. [Online]. Available: <https://www.cs.utexas.edu/~EWD/ewd01xx/EWD123.PDF>
- [4] The Open Group, “sem_overview(7) — overview of POSIX semaphores,” *Linux man-pages*. [Online]. Available: https://man7.org/linux/man-pages/man7/sem_overview.7.html
- [5] The Open Group, “pthreads(7) — POSIX threads,” *Linux man-pages*. [Online]. Available: <https://man7.org/linux/man-pages/man7/pthreads.7.html>
- [6] The Open Group, “pthread_mutex_lock(3) — lock and unlock a mutex,” *Linux man-pages*. [Online]. Available: https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html
- [7] A. B. Downey, *The Little Book of Semaphores*, 2nd ed. Green Tea Press, 2016. [Online]. Available: <https://greenteapress.com/wp/semaphores/>